

Использование языка программирования Julia для решения математических задач

Часть I

Авторы:

Чуракова Юлия Романовна

*Студентка факультета математики и компьютерных наук
Кубанского Государственного Университета*

Алексеев Евгений Ростиславович

*Кандидат технических наук, доцент
Доцент кафедры информационных технологий
Кубанского Государственного Университета*

Полная версия пособия доступна в репозитории:

<https://github.com/JuliaChurakova/Julia>

Литература

1. Энгхейм, Э. Julia в качестве второго языка / Э. Энгхейм. – Москва: ДМК Пресс, 2023. – 446 с.
2. The Julia Programming Language: сайт. – URL: <https://julialang.org> (дата обращения: 10.10.2025).
3. Белов, Г. В. Краткое описание языка программирования Julia и некоторые примеры его использования/ Г. В. Белов. – Москва: МГТУ им. Н.Э. Баумана, 2024. – 108 с.
4. Шеррингтон, М. Осваиваем язык Julia / М. Шеррингтон. – Москва: ДМК Пресс, 2017. – 416 с.
5. Антонюк, В. А. Язык Julia как инструмент исследователя / В. А. Антонюк. – Москва: Физический факультет МГУ им. М. В. Ломоносова, 2019. – 48 с.

Глава 5 Примеры программ на языке Julia

ЗАДАЧА 1. Найти первые **M** чисел Армстронга. Число Армстронга – натуральное число, равно сумме своих цифр, возведённых в степень, равную количеству его цифр.

Мы решили проверить скорость работы двух программ, написанных на Python и Julia, которые находят и выводят первые M чисел Армстронга.

Код на Python:

```
import time

def armstrong(x):
    p = x
    x_str = str(x)
    l = len(x_str)
    sum_x = 0
    while x != 0:
        sum_x += (x % 10) ** l
        x = x // 10
    if sum_x == p:
        return True
    else:
        return False

def find_armstrong_numbers(M):
    m = 0
    x = 0
    while m < M:
        x += 1
```

```

        if armstrong(x):
            m += 1
            print(f"{m}) x={x}")
        print("Цикл завершился!")

print("Введите количество чисел Армстронга, которые хотите найти: ")
M = int(input())
start_time = time.time()
find_armstrong_numbers(M)
end_time = time.time()
print(f"Время выполнения: {end_time - start_time:.6f} секунд")

```

Результат работы программы

```

Введите количество чисел Армстронга, которые хотите найти: 20
1) x=1
2) x=2
3) x=3
4) x=4
5) x=5
6) x=6
7) x=7
8) x=8
9) x=9
10) x=153
11) x=370
12) x=371
13) x=407
14) x=1634
15) x=8208
16) x=9474
17) x=54748
18) x=92727
19) x=93084
20) x=548834
Цикл завершился!
Время выполнения: 0.823005 секунд

```

```

In [16]: function armstrong(x)
        p = x
        x_str = string(x)
        l = length(x_str)
        sum_x = 0
        while x != 0
            sum_x += (x % 10)^l
            x = x ÷ 10
        end
        if sum_x == p
            return true
        else
            return false
        end

```

```

end

function find_armstrong_numbers(M)
    m = 0
    x = 0
    while m < M
        x += 1
        if armstrong(x)
            m += 1
            println("$m x=$x")
        end
    end
    println("Цикл завершился!")
end

println("Введите количество чисел Армстронга, которые хотите найти")
M = parse{Int, readline()}
println("Введенное значение: ", M)
start_time = time()
find_armstrong_numbers(M)
end_time = time()
println("Время выполнения: $(end_time - start_time) секунд")

```

Введите количество чисел Армстронга, которые хотите найти

Введенное значение: 20

- 1) x=1
- 2) x=2
- 3) x=3
- 4) x=4
- 5) x=5
- 6) x=6
- 7) x=7
- 8) x=8
- 9) x=9
- 10) x=153
- 11) x=370
- 12) x=371
- 13) x=407
- 14) x=1634
- 15) x=8208
- 16) x=9474
- 17) x=54748
- 18) x=92727
- 19) x=93084
- 20) x=548834

Цикл завершился!

Время выполнения: 0.09553098678588867 секунд

Julia продемонстрировала значительно более высокую производительность по сравнению с Python. Время выполнения программы на Julia для нахождения и вывода первых M чисел Армстронга было заметно меньше, что свидетельствует о высокой эффективности и оптимизации языка. Это делает Julia предпочтительным выбором для задач, требующих высокой скорости работы.

ЗАДАЧА 2. Решить уравнения методом Ньютона-Рафсона.

Метод Ньютона-Рафсона – итерационный алгоритм поиска корней уравнений вида $f(x) = 0$.

```
In [46]: function newton_raphson(f, df, x0, tol=1e-7, max_iter=1000)
    # Инициализация начального значения
    x = x0

    # Главный итерационный цикл
    for i in 1:max_iter
        # Вычисляем значение функции в текущей точке
        fx = f(x)

        # Критерий остановки: достигнута требуемая точность
        if abs(fx) < tol
            return x # Возвращаем найденный корень
        end

        # Вычисляем значение производной
        dfx = df(x)

        # Проверка на нулевую производную (метод не применим)
        if dfx == 0
            error("Производная равна нулю, метод не применим.")
        end

        # Основная формула метода Ньютона
        x = x - fx / dfx # Получаем следующее приближение
    end

    # Если превышено максимальное число итераций
    error("Максимальное количество итераций достигнуто, решение не найдено.")
end

# Определяем тестовую функцию
f(x) = x^2 - 2

# Аналитическая производная функции
df(x) = 2x

# Начальное приближение
x0 = 1.0

# Вызываем метод Ньютона-Рафсона
root = newton_raphson(f, df, x0)

println("Корень уравнения x^2 - 2 = 0: ", root)
```

Корень уравнения $x^2 - 2 = 0$: 1.4142135623746899

ЗАДАЧА 3. Вывести на экран первую тысячу чисел Хэмминга. Число Хэмминга – это положительное целое число вида $2^i \cdot 3^j \cdot 5^k$ для некоторых неотрицательных целых чисел i, j , и k . Первое число Хэмминга равно

$1 = 2^0 \cdot 3^0 \cdot 5^0$, второе число Хэмминга равно $2 = 2^1 \cdot 3^0 \cdot 5^0$, третье число Хэмминга равно $3 = 2^0 \cdot 3^1 \cdot 5^0$, четвертое число Хэмминга равно $4 = 2^2 \cdot 3^0 \cdot 5^0$, пятое число Хэмминга равно $5 = 2^0 \cdot 3^0 \cdot 5^1$.

```
In [17]: # Функция проверки, является ли число числом Хемминга
function heming(n)
    # Делим число на 5, пока это возможно
    while n % 5 == 0
        n = n / 5
    end

    # Затем делим на 3, пока это возможно
    while n % 3 == 0
        n = n / 3
    end

    # Затем делим на 2, пока это возможно
    while n % 2 == 0
        n = n / 2
    end

    # Если после всех делений получили 1 - это число Хемминга
    return n == 1
end

# Основная программа для поиска первых k чисел Хемминга

# Ввод количества чисел Хемминга, которые нужно найти
println("Введите количество чисел Хемминга, которые хотите найти")
k = parse{Int64, readline()}
println("Введенное значение: ", k)

count = 0 # Счетчик найденных чисел Хемминга
j = 1     # Текущее проверяемое число

# Ищем числа Хемминга, пока не найдем нужное количество
while count < k
    if heming(j)
        # Если число Хемминга - увеличиваем счетчик и выводим
        count += 1
        println("№=", count, ", ", j)
    end
    j += 1 # Переходим к следующему числу
end
```

Введите количество чисел Хемминга, которые хотите найти

Введенное значение: 20

$M_2=1, 1$
 $M_2=2, 2$
 $M_2=3, 3$
 $M_2=4, 4$
 $M_2=5, 5$
 $M_2=6, 6$
 $M_2=7, 8$
 $M_2=8, 9$
 $M_2=9, 10$
 $M_2=10, 12$
 $M_2=11, 15$
 $M_2=12, 16$
 $M_2=13, 18$
 $M_2=14, 20$
 $M_2=15, 24$
 $M_2=16, 25$
 $M_2=17, 27$
 $M_2=18, 30$
 $M_2=19, 32$
 $M_2=20, 36$

ЗАДАЧА 4. Найти сумму квадратов первых n ($100 \leq n \leq 1000$) чисел, кратных 7.

```
In [18]: function sum_squares(n)
          first = 7
          sum = 0
          for i in 0:(n-1)
              num = first + i * 7
              sum += num ^ 2
          end
          return sum
        end

println("Введите количество чисел, для которых нужно найти сумму квадратов (
n = parse{Int, readline()}
println("Введенное значение: ", n)
if 100 <= n <= 1000
    result = sum_squares(n)
    println("Сумма квадратов первых $n чисел, кратных 7: $result")
else
    println("Ошибка: n должно быть в диапазоне от 100 до 1000.")
end
```

Введите количество чисел, для которых нужно найти сумму квадратов ($100 \leq n \leq 1000$):

Введенное значение: 234

Сумма квадратов первых 234 чисел, кратных 7: 210620865